

Introduction to Quantum Computing

COMP2048 Lecture Notes, Riddhi S. Gupta

April 22, 2026

Contents

1	Overview	2
1.1	Quantum computers as Turing machines	3
1.2	Quantum and classical complexity	6
2	Quantum postulates	9
2.1	Postulate 1: Quantum states	10
2.2	Postulate 2: Quantum operations	13
2.3	Universality of quantum computing	17
2.4	Postulate 3: Born's rule	17
2.5	Postulate 4: Compositions of quantum systems	19
3	Extra for experts (not examinable)	21
3.1	Postulates revisited	21
3.2	Noisy quantum states	24
3.3	No cloning theorem	25
4	Quantum algorithms	28
4.1	Constant vs. balanced oracle problem	28
4.2	Phase kickback	29
4.3	Deutsch's algorithm	31
4.4	Extending to Deutsch-Jozsa algorithm	33
4.5	Random classical algorithm	37
4.6	Bernstein-Varizani problem (not examinable)	37

The theory of computation has traditionally been studied almost entirely in the abstract, as a topic in pure mathematics. This is to miss the point of it. Computers are physical objects, and computations are physical processes. What computers can or cannot compute is determined by the laws of physics alone, and not by pure mathematics. - David Deutsch

1 Overview

What is the physicality of computation? The possibility that we can leverage quantum mechanics to do computation in new and interesting ways could, theoretically, form the basis of a very powerful form of computation. These set of notes are to company an introduction to quantum computing in three lectures for COMP2048. The main ideas we will cover are presented in introductory quantum computing courses for non-physicists.

I'm afraid our common-sense intuition about the physical world around us doesn't apply to quantum computing, and so we need to rely on precise tools to articulate our thoughts. We will go through complex numbers, vectors and matrices in the first lecture so that we have enough tools to understand a simple quantum algorithm - Deutsch's algorithm. If you haven't come across complex numbers, vectors or matrices before, it might be useful to look up some tutorials prior to attending lectures. Additionally, I will try to use symbolic notation, called Dirac notation, that will reduce the need to work with matrices or vectors.

No mathematical derivations will be required in the exam but exercises marked with * will be examinable in assignments and tutorials. Meanwhile, some of you come from strong linear algebra backgrounds and you might find handwavy expositions unsatisfying. For this audience, I have provided more precise definitions in Section 3 and Section 3 is not examinable. You can read Section 4 and skip Section 3 entirely if you prefer.

I will explain relevant ideas we need to introduce quantum computing along the way. To this end, you might find it useful to read Chapter 1 of Ref. [1]. Another introductory book written for engineers that tries to bridge this gap is Ref. [2] (with associated code available in Git [3]). If you would like to pursue quantum computing at a graduate level, you might want to check out John Watrous' textbook [4] and an excellent set of Youtube lectures [5] with codebase in a Pythonic library, qiskit. Meanwhile, Refs.[6, 7] cater to candidates who prefer mathematical or physics explanations of quantum mechanics with applications to computing.

1.1 Quantum computers as Turing machines

In 1936, Alan Turing described an abstract model of computation, called a Turing machine, that we would now recognise as a modern, general-purpose programmable computer. He claimed that any algorithmic task could be performed by a Universal Turing Machine, and this machine could simulate any other Turing machine.

The next several decades saw rapid advancements in the development of physical hardware that could perform generally programmable computations. A leading hardware platform materialized whereby electronic switches represented discrete bits of information. This computing hardware could be contrasted with analogue computing, for example. In any case, circuit-based hardware for electronic computations improved a pace that has come to be called Moore's law, whereby computer power doubled for constant cost roughly once every two years [1], at least until very recently.

Now however, physical systems for electronic computation appears to be limited in the sense that further reductions in size may not be accessible without having to take into account the physics of small systems i.e. quantum mechanical effects. Perhaps we could adjust our design or engineering calculations to accommodate any errors arising from treating a system according to classical physics even though its behavior might be better described quantum mechanically.

Or perhaps we might be able to process information - not according to the classical physics of our everyday lives - but instead according to new physical rules that only apply when systems become very small (and reasonably well isolated) from their surroundings. Could current limits on physical computation be overcome by switching to an entirely new paradigm of computing? Addressing this tantalizing question is at the heart of theoretical and experimental quantum computing today.

For our purposes, we will informally define such a device where its components behave according to the rules of quantum mechanics.

Definition 1 (A quantum computer). *A quantum computer is a device that uses the principles of quantum mechanics to perform computation.*

Notice that any such quantum computing device is still describable as a Turing machine. This correspondence between the evolution of a quantum system and a Universal Turing Machine was first outlined in the 1980s, where Ref. [8] argued that a computation that could be mapped to quantum mechanical states and operators is equivalent to a Turing machine of equivalent or greater power [2]. The first definition of quantum Turing ma-

chines (QTMs) was provided in 1985 by David Deutsch [9], where such a machine has an infinite tape, a transition function, and the actions of the machine are local and completely specified by this transition function. However, quantum Turing machines have two additional departures: first, the machine's evolution is always time reversible, and secondly, at each step, transition probabilities can *interfere*, i.e. amplify or cancel out, according to the rules of quantum mechanics. Subsequently, it was shown that QTMs are discrete machines even though transition probabilities are real valued and interference is typically thought of in terms of continuous behavior [10].

Since we know that quantum computation can be seen as Turing machines, we know that quantum computing conforms to the Church-Turing conjecture, i.e. from Week 5's lectures,

Definition 2 (Church-Turing Thesis CTT). *Any function that can be computed by an algorithm can be computed by a Turing Machine.*

This statement about computability. However, the discovery of certain quantum algorithms appear to challenge the Extended Church-Turing Thesis, which is statement about *efficiency* of computation.

Definition 3 (Extended Church-Turing Thesis ECTT). *Any algorithmic process can be simulated efficiently using a Probabilistic Turing Machine. [2]*

This thesis asserts that any reasonable computational model can be simulated efficiently by a probabilistic Turing machine. Here, efficiency means the simulation overhead is bounded by a polynomial. You may have already heard that the class of problems solvable in polynomial time by classical computers is P (deterministic bits) or BPP (random bits). It is known that $P \subseteq BPP$, but it is unknown if $P = BPP$. The ECTT conjectures that all physically realizable computations belong to BPP . The implication is that no physical machine operating according to the rules of everyday physics, i.e. classical computers, can solve problems much faster than probabilistic Turing machines.

But what do we mean by 'physically realizable algorithmic process'? What if the computers that we build do not operate according to the rules of classical physics? What if we instead use the rules of quantum physics to process information? Remarkably in the 1980s and 1990s, it was found that algorithms that processed information according to the rules of quantum physics could solve certain problems exponentially faster than algorithms behaving according to the rules of classical physics. Some of these early

discoveries are give in Table 1. David Deutsch first demonstrated that a quantum computer could solve a task requiring two classical resources in a single quantum step. Later, this problem was extended to the Deutsch-Josza problem which showed that a quantum computer could solve a task requiring exponential classical resources in a single step. Subsequent improvements culminated in Peter Shor’s 1994 demonstration that two enormously important problems - finding prime factors of integers and the discrete log problem - could be efficiently solved by quantum computers. That the former problem cannot be solved efficiently by classical computers forms the basis of modern encryption techniques (e.g. RSA). Indeed the existence of an efficient quantum algorithm stimulated interest in both quantum computing and quantum-safe cryptography. These, and other potential speed advantages in search, linear algebra, and physical simulations, are so significant that many researchers believe that no conceivable amount of progress in classical computation would be able to overcome the gap between the power of a classical computer and the power of a quantum computer.

QA	Year	Problem	Classical	Quantum	Speedup
Deutsch	1985	Determine if Boolean function is constant or balanced (1-bit case)	2 queries	1 query	Factor of 2
Deutsch-Jozsa	1992	Determine if function is constant or balanced	$2^{n-1} + 1$ queries (deterministic)	1 query	Exp.
Bernstein-Vazirani	1993	Find hidden bit string s	n queries	1 query	Linear
Simons	1994	Find hidden XOR mask s	$O(2^{n/2})$ queries	$O(n)$ queries	Exp.
Shor	1994	Integer factorization / discrete log	Sub-exponential $e^{(\log n)^{1/3}}$	Polynomial $O((\log n)^3)$	Exp.
Grover	1996	Unstructured search	$O(N)$	$O(\sqrt{N})$	Quadratic

Table 1: Important quantum algorithms from the 1980s–1990s and their classical vs quantum complexities. A modern course on quantum algorithms can be found in Ref.[11], and an online compendium of all proposals is given in Ref. [12].

Early proofs of quantum advantage can be understood using a *query*

model of computation, which is a framework for analyzing computation, rather than how computation is actually performed in practice. Here one assumes you have access to an *oracle* to gain information about a problem. The notation $\mathcal{O}(\cdot)$ conveys the worst-case upper bound to the number of times an oracle is queried to solve a task. This scaling behaviour of algorithms is compared when the oracle is either *classical* or *quantum*. Informally we can say that an algorithm is *inefficient* if runtime grows exponentially in problem / input size.

The discovery of early quantum algorithms in Table 1 undermined the ECTT assertion that any universal turning machine or even Probabilistic Turing Machine could simulate any other computing device efficiently. As such, the ECTT was reformulated to the Quantum ECTT,

Definition 4 (Quantum Extended Church Turing Thesis, [2]). *Any realistic physical computing device can be efficiently simulated by a fault-tolerant quantum computer.*

This statement should be thought of as a thesis or a conjecture rather than a statement of fact.

Formally, the distinctions between efficient and inefficient algorithms are made mathematically precise by the field of computational complexity, which you have covered in previous lectures, where algorithms with polynomial runtime are considered efficient. With the discovery of quantum algorithms, new quantum complexity classes aim to better distinguish problems that quantum computers can solve, and how these classes of problems fit into the known complexity hierarchy for classical computation. We recap classical complexity classes and some quantum complexity classes in the next subsection.

1.2 Quantum and classical complexity

A summary of complexity classes is provided in Fig. 1 summarising what is known, as well as open questions in the field.

Classically, Fig. 1a depicts known relationships between classical complexity classes discussed in Ref. [13], where each line depicts containments going upwards. I will not define or discuss each complexity class but I will informally summarise them as follows. The class P is the class of computational problems that can be solved quickly on a classical computer. NP is the class of problems which have solutions which can be quickly checked on a classical computer. NP -complete problems are ‘at least as hard’ as all other problems in NP . PSPACE consists of those problems which can be

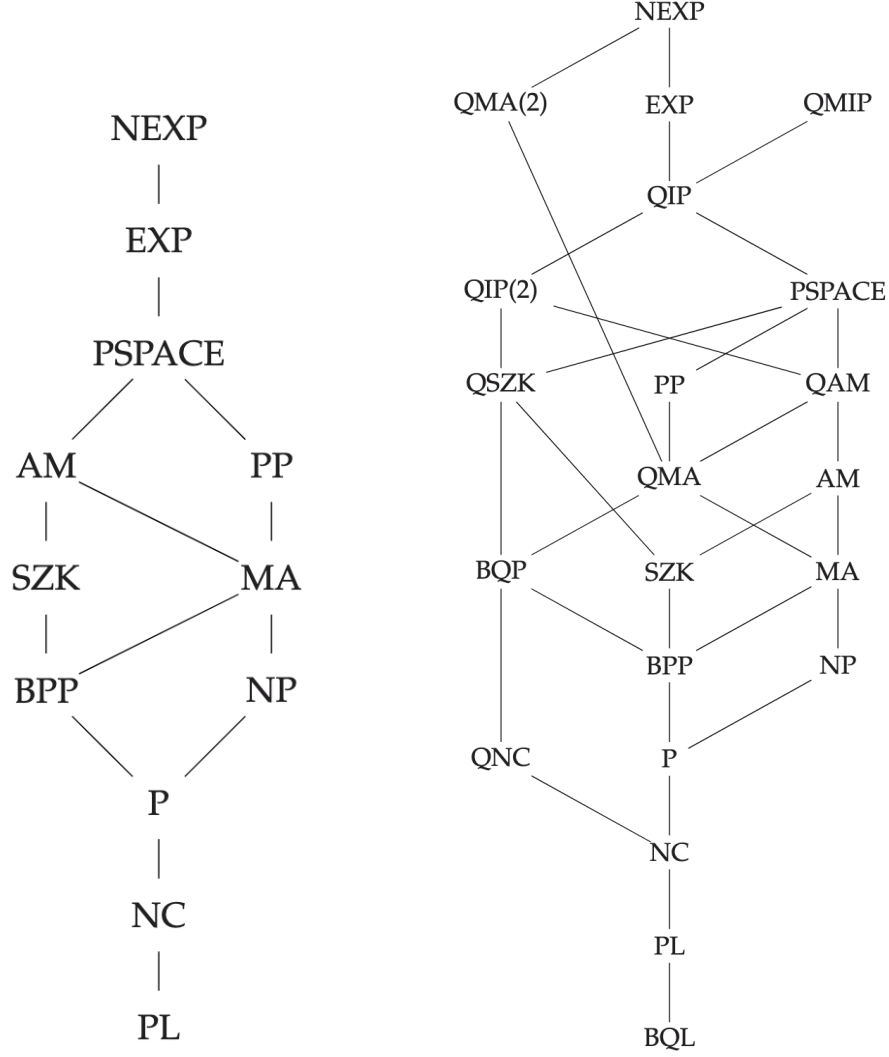
solved using resources which are few in spatial size but not necessarily in time. Currently there is no proof that $P = NP$. If $P \neq NP$, then no NP -complete problem can be efficiently solved on a classical computer. Finally, the complexity class BPP is the class of problems that can be solved using randomized algorithms in polynomial time with a bounded probability of error. BPP is widely regarded as being the class of problems which should be considered efficiently solvable on a classical computer. It is not known if $BPP \subseteq NP$.

Quantumly, Fig. 1b depicts a number of quantum complexity classes, but we will only discuss BQP . Informally, BQP is the class of all computational problems which can be solved efficiently on a quantum computer with a bounded probability of error. It is considered to be an analogue of BPP . The key point is that where BQP contains P , BPP but it's relationship to NP is as yet unknown.

Can we prove that quantum computers are more efficient than classical computers? This statement would require the proof that $BPP \neq BQP$. From Fig. 1b, we notice that $P \subseteq BPP \subseteq BQP \subseteq PP \subseteq PSPACE$. Proving that $BPP \neq BQP$ implies that $P \neq PSPACE$ which is as hard as proving $P \neq NP$. If it is proved that quantum computers are strictly more powerful than classical computers, then it will follow that P is not equal to $PSPACE$.

Note that while containments between classes are known, i.e. , $P \subseteq BPP \subseteq BQP \subseteq PP \subseteq PSPACE$, we don't yet know if these classes increase or are exactly equivalent. Consider, for example, $BPP \subseteq BQP$. Here, factoring is believed to be in BQP but hoped not to be in BPP . This conjecture forms the basis of RSA encryption techniques. Specifically, factoring is not (currently known to be) NP -complete and it is believed to be in NP but not in P . Indeed, it is not known whether quantum computers can be used to quickly solve all the problems in NP . A introductory discussion of quantum complexity classes can be found in Refs. [2, 1] and a detailed discussion can be found in [13].

Our goal will be to understand the nature of quantum advantage through specific promise problems. We will focus on the first two algorithms in Table 1, namely the Deutsch Algorithm in lectures, and Deutsch-Josza Algorithm in a Python based lab. It turns out that the Bernstein-Vazirani can also be solved by the Deutsch-Josza Algorithm, as we will explore in the lab. What we cannot cover is that many problems with exponential advantage in Table 1 have a specific, periodic structure that quantum algorithms exploit better than classical algorithms. Indeed the tasks in Table 1 can be seen



(a) Classical complexity classes.

(b) Quantum and classical complexity.

Figure 1: Complexity containments discussed in Ref. [13]. Lines depict known containments moving upwards, e.g. $P \subseteq NP$. (a) classical complexity classes, for reference, to be compared with (b) quantum complexity classes and their relationships to classes in (a). Of these, BQP (bounded-error quantum polynomial time) are the class of problems efficiently solvable on a quantum computer.

as special cases of a general problem, known as the hidden sub-group problem. Other speed advantages, such as Grover’s algorithm for unstructured search, and algorithms like HHL for linear algebra, or simulations of physical phenomena will not be discussed here but to which Ref. [1] provides an introduction.

Restricting our attention to the simplest version of quantum algorithms, we first talk about the rules, or postulates, of quantum computation which these algorithms use to process information.

2 Quantum postulates

A postulate is a statement that is accepted as true without proof and used as a starting assumption for constructing theories or mathematical systems. Quantum theory refers to a set of postulates or rules by which we describe the behaviour of physical systems. We observe that physical systems behave according to the rules of quantum mechanics when a physical system is very well-isolated from the rest of the universe. Physically, we can achieve this in special laboratories where we carefully observe physical systems that are e.g. very small, very cold, and/or in-vacuum. (Analogously, Einstein’s rules of special relativity don’t apply to an observer at rest, but only to observers that are moving close to the speed of light.) A quantum algorithm processes information according to the rules of quantum mechanics in order to solve particular problem.

Exercise: Consider a coin in your hand. Why doesn’t this coin behave quantum mechanically?

Let’s now go through each of the postulates that will tell you what these rules are, and how we use them to design quantum algorithms. Informally, the goal of a quantum algorithm is to prepare a quantum state (Postulate 1) using transformations (Postulate 2) that when this state is measured (Postulate 3), the observed outcomes will be sampled from a probability distribution that will solve a task with high degree of certainty. When we scale the size of the problem to many quantum systems (Postulate 4), then quantum advantage is achieved if there is no classical problem that can efficiently solve the same problem.

2.1 Postulate 1: Quantum states

Under one interpretation of quantum mechanics, a quantum state contains all information about a physical system to derive probabilities for measurements performed on the system.

Postulate 1 (Quantum states.). *A quantum state contains all information about a physical system to derive probabilities for measurements performed on the system. We cannot directly observe a quantum state but it provides a description from which probability distributions of any measured outcomes can be derived.*

A remarkable assertion in quantum mechanics is that all possible information about physically measurable properties of a quantum system are contained in this abstract quantum state. The quantum state contains information about the probabilities of all possible outcomes we can observe if we were to perform experiments. We cannot directly observe the quantum state, but only compute probabilities of observations from the information contained within it.

While sounding mysterious, a quantum state is represented concretely by mathematical objects. Mathematically, we say that if our system or a machine has a finite number of discrete states N , then our quantum state will be a N -dimensional *complex* unit vector.

Why do we need to use complex numbers to describe quantum states? Well in other areas of engineering and science, complex numbers help us to describe the behavior of waves (sound waves, water waves etc.) and phenomena like superpositions of waves and wave interference.

However, in quantum mechanics, we use complex numbers to describe the interference and superpositions of *quantum states* of a system. Physically, it implies that the behavior of fundamental particles we observe in experiments breaks the distinction between waves and particles, e.g. an electron can behave both like a wave or a particle.

Definition 5 (Complex numbers). *A complex number is a number that has a real part and an imaginary part. We denote the imaginary part $i = \sqrt{-1}$. Let α, β be 2 complex numbers $\alpha = a + bi$ and $\beta = c + di$ where a, b, c, d are real. Then we can:*

- Add complex numbers, $\alpha + \beta = (a + c) + (b + d)i$
- Multiply complex numbers, $\alpha\beta = (a+bi)(c+di) = (ac-bd)i+(ad+bc)i$
- Conjugate complex numbers $\alpha^* := a - bi$ by flipping the sign next to i

- Get mod squared $|\alpha|^2 := \alpha^* \alpha = (a + bi)(a - bi) = a^2 + b^2$
- Represent a complex number using a amplitude and a phase, $\alpha = |\alpha|e^{i\theta}$, $\theta := \arctan(b/a)$

For finite N , we can mathematically represent this wavefunction as an array of complex numbers. We call a vector of complex numbers a quantum state, a wave-function or a vector of probability amplitudes,

$$\begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{N-1} \end{bmatrix}, \quad v_j \in \mathbb{C}, \forall j, \quad (1)$$

and since this is a *unit* vector, we require that $\sum_j |v_j|^2 = 1$.

Definition 6 (Qubit). *A quantum bit (qubit) is the special case where $N = 2$.*

Physically a qubit can be associated to the ground (lowest energy) state and an excited state of a system, such as two atomic energy levels. The quantum state for a qubit is described by two complex numbers, α, β ,

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad (2)$$

such that $|\alpha|^2 + |\beta|^2 = 1$.

In Dirac notation, a quantum state is denoted as ket, $|\cdot\rangle$. So a general state of the qubit, call it $|\Psi\rangle$, can be defined as,

$$|\Psi\rangle := \begin{bmatrix} \alpha \\ \beta \end{bmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (3)$$

Now, in writing this quantum state, we've already assumed a basis,

$$|\Psi\rangle = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (4)$$

We will represent $|0\rangle$ and $|1\rangle$ as the basis vectors $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ and $\begin{bmatrix} 0 \\ 1 \end{bmatrix}$, which gives

$$|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (5)$$

If we can write $|\Psi\rangle$ as a (linear) combination of other states, we call this combination a quantum superposition.

Definition 7. *Superposition: Any quantum state can be expanded as a convex, complex linear combination of other states, called a superposition.*

Here, convex means that the absolute value of complex numbers squared sum to one $|\alpha|^2 + |\beta|^2 = 1$, and linear means that you can multiply quantum states with a complex number and add quantum states together like $\alpha|0\rangle + \beta|1\rangle$ in Eq. (5).

For every quantum state, we can also associate an object called a bra $\langle \cdot |$. This is just a row vector of the conjugate complex numbers,

$$\langle \Psi | := [\alpha^* \quad \beta^*]. \quad (6)$$

This notation represents ‘bra-ket’ or Dirac notation. The usefulness of these bra-vectors is that you can combine them with ket-vectors via vector multiplication, and this gives you a complex scalar. For example, all quantum states have a length of one, and you can see this by,

$$\langle \Psi | \Psi \rangle = [\alpha^* \quad \beta^*] \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = 1. \quad (7)$$

Notice, that this must be unity.

Exercise: why is $\langle \Psi | \Psi \rangle = 1$?

Physically, this the bra-ket notation help us to compute the overlap between two quantum states. Suppose we have two quantum states, such as $|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle$ and $|\Phi\rangle = \zeta|0\rangle + \gamma|1\rangle$, where $\alpha, \beta, \gamma, \zeta$ are all complex numbers and both states have a length of one, i.e. $|\alpha|^2 + |\beta|^2 = 1$ and $|\zeta|^2 + |\gamma|^2 = 1$. Then the bra-ket (braket) of these two states is,

$$\langle \Phi | \Psi \rangle = [\zeta^* \quad \gamma^*] \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \zeta^* \alpha + \gamma^* \beta. \quad (8)$$

We have already seen that this quantity is one if two quantum states are identical. You can also verify that this quantity is zero if two quantum states have no overlap e.g., if $\alpha = 0, \gamma = 0$.

Exercise: Compute $\langle 0|0\rangle, \langle 0|1\rangle, \langle 1|0\rangle, \langle 1|1\rangle$ to show that the kets $|0\rangle, |1\rangle$ either have perfect overlap or no overlap.

2.2 Postulate 2: Quantum operations

In this subsection, we describe how to transform quantum states to other valid quantum states. What this means is that you can break up your quantum algorithm into a sequential series of logical operations you want to perform on your input state. If a quantum state is represented by a vector, you might have guessed that these quantum operations will be presented by a matrix.

Before we go further, let us remind ourselves what a matrix is.

Definition 8. *A matrix is a $N \times M$ array of (complex) numbers. Matrices can be square ($N = M$) or rectangular, but we will work with square matrices. Here's a short list of what you can do with them:*

- *Multiply by a complex number*
- *Add two matrices of the same dimension*
- *Multiply two matrices AB if number of columns of A equals number of rows of B .*
- *Multiply a matrix and a column vector (or a row vector) from the right $A|\Psi\rangle$ (or left, $\langle\Psi|A$)*
- *Conjugate all entries of a matrix A^**
- *Transpose a matrix A^T*
- *Do conjugation then transpose, or transpose then conjugation, $A^\dagger = (A^*)^T = (A^T)^*$*
- *Some square matrices have an inverse, such that $AA^{-1} = A^{-1}A = I$. If A does not have an inverse, it is called singular*

One important fact is that order of matrix multiplication matters. In general AB does not equal BA . If $AB = BA$ we say A and B commute.

We now describe that all transformations of a quantum state are given by operators or matrices of a very specific type.

Postulate 2. *The evolution in time of our description of the device is reversible i.e. a unitary matrix acting on a state vector transforms from $|\Psi\rangle \rightarrow U|\Psi\rangle$ where U is a matrix satisfying $U^\dagger U = UU^\dagger = I$. (Alternatively, $U^{-1} = U^\dagger$.)*

For now, just know that common single-qubit gates are the (unitary) matrices,

Single Qubit Gate	Matrix
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$

Exercise: Show that the X gate negates a quantum bit i.e. compute $X|0\rangle$ and $X|1\rangle$. Which classical logical operator does this correspond to?

Exercise: Show that $|+\rangle := H|0\rangle$ and $|-\rangle := H|1\rangle$ are a superposition of $|0\rangle$ and $|1\rangle$ quantum states.

Exercise: Show that you can write $|0\rangle$ as a superposition of $|+\rangle$ and $|-\rangle$.

Exercise: Show that $HXH = Z$ and $HZH = X$.

So far we have discussed that the time evolution of a quantum state in time must be reversible (i.e. unitary). All single qubit gates listed above are reversible. While we haven't talked about two-qubit systems yet, you can confirm that these matrices are also unitary. Later on, we will use these gates to create interactions between two qubits.

Two-Qubit Gate	Matrix
CX	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$
CZ	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}$

However, not all quantum operations are unitary. For example, the action of measurements are irreversible actions which we will cover by the next postulate.

It's a good time to notice that we can write gates using the outer products of bras and kets. Previously, we used bra-ket notation to compute an inner product of vectors i.e. a row vector multiplied by a column vector of equal length, which gave us a *complex number*. We can also multiply two vectors to give us a *matrix*.

Definition 9 (Outer product). *An outer product between two column vectors v, w is given by multiplying the column vector w with the transpose of v^* resulting in a matrix. In Dirac bra-ket notation, we just write $|w\rangle\langle v|$.*

Exercise: Compute the outer products $|0\rangle\langle 0|$, $|1\rangle\langle 1|$. Then show that you can write the following single-qubit gates as outer products,

$$I = |0\rangle\langle 0| + |1\rangle\langle 1|, \quad (9)$$

$$Z = |0\rangle\langle 0| - |1\rangle\langle 1|, \quad (10)$$

$$X = |+\rangle\langle +| - |-\rangle\langle -|. \quad (11)$$

Establishing the identities above should make it easier to do quantum computation without fully writing out matrices and vectors.

Finally the single qubit gates $\{X, Y, Z\}$ allow us to visually depict a qubit like a sphere. A detailed derivation is provided in Section 3.2 but for now, all we need to know is that these matrices define a coordinate system and all noiseless qubit quantum states are points on the surface of a unit sphere. In Fig. 2, I show this sphere, called a Bloch sphere, where the states $|0\rangle$ and $|+\rangle$ are drawn as arrows from the origin to the surface of the sphere. The action of the H gate is shown as the blue arc. Other single qubit rotations can rotate a quantum state to other points on the surface of the sphere.

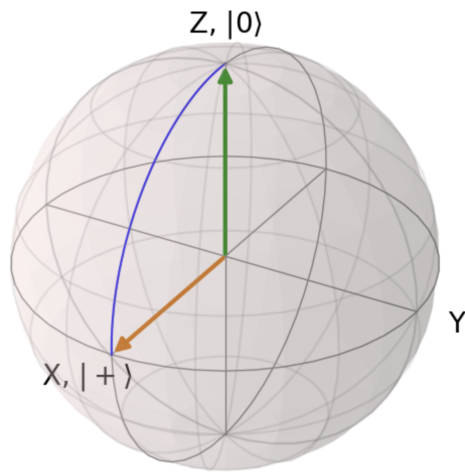


Figure 2: Bloch sphere: the coordinate system represents directions along X , Y and Z Pauli operators. The $|0\rangle$ state (green) corresponds to the vector $[0, 0, 1]$ and $|+\rangle$ (brown) corresponds to the vector $[1, 0, 0]$. The blue arc represents the action of an H gate. Every *pure* state of a qubit is on the surface of this sphere, while every mixed (noisy) quantum state is in the bulk of the sphere.

2.3 Universality of quantum computing

In classical computing we have a set of commonly used gates: AND, NOT, OR, NAND, XOR, FANOUT, etc. We can use combinations of these gates to perform any computation in classical computing. A classical computer that can run these gates is Turing-complete or universal.

Is there a gate or set of gates that are universal for quantum computing? In fact, there are several combinations of gate sets that lead to universality. Of these, the gate set $\{CX, T, S\}$ is universal and consists of 2 single qubit gates, and 1 two qubit gate, from which any N -qubit circuit can perform computation. For each hardware platform, a quantum computer will natively (physically) implement some kind of two-qubit interaction. This physical interaction is then converted to a logical CX gate by pre- or post-multiplying the interaction with single-qubit gates.

There are two other gate sets that are worth mentioning in quantum computing. The first is the Clifford group, generated by $\{CX, H, S\}$. It is known that any computation decomposable into elements of the Clifford group. The second is the Pauli group, which is the name given to the set of single qubit gates $\{I, X, Y, Z\}$. The two groups are related since for a Clifford operator C and any Pauli P , the product $C^\dagger P C$ gives you another Pauli operator Q . Notice that multiplying any two Pauli gates gives you another Pauli gate, so the Pauli group is a subset of the Clifford group.

Any quantum computation that contains only Clifford group elements is efficiently classically simulable. To make the Clifford group more interesting, we add a T gate, and it is known that the Clifford + T group is a universal gate set. As an illustration, I've written some code so that you can see how one can rotate a $|0\rangle$ (green) to other pure states (blue points) through random sequences of varying lengths using only H and T gates (see Fig. 3). A specific single qubit gate can indeed be decomposed into H and T gates to arbitrary precision but these sequences will not be random.

In the next section, we introduce quantum measurements. Unlike quantum gates we have just discussed, measurements represent irreversible (non unitary) transformations of quantum states.

2.4 Postulate 3: Born's rule

So far we have talked about quantum states (vectors), and how you can use quantum operators (matrices) to transform them. What we haven't talked about is how to get information out of a quantum computer. We call this process, measurement, and the next postulate addresses this.

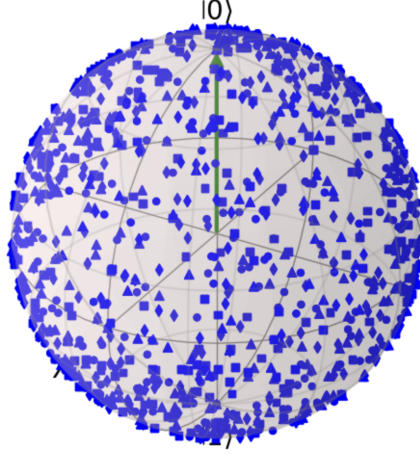


Figure 3: Transforming a $|0\rangle$ (green) to other pure states (blue points) through random sequences of varying lengths to a maximum of 1000 gates, but using only H and T gates. Note that $T^2 = S$.

Postulate 3 (Born’s rule (for a single qubit)). *Suppose we write our qubit state using Eq. (5),*

$$|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1. \quad (12)$$

Then the probability of measuring 0 is $|\alpha|^2$ and we measure 1 with probability $|\beta|^2$, which is incidentally the probability $1 - |\alpha|^2$.

If we measure ‘0’, we say that the system which was in state $|\Psi\rangle$ has ‘collapsed’ to the state $|0\rangle$. If we measure ‘1’, we say that the system which was in state $|\Psi\rangle$ has ‘collapsed’ to the state $|1\rangle$. The term collapse refers to the fact that $|\Psi\rangle$ is no longer a superposition of ‘0’ and ‘1’. If we hadn’t performed the measurement, the system would have remained in the superposition $|\Psi\rangle$ but we forced it to randomly choose one of the states $|0\rangle$ or $|1\rangle$ with probability $|\alpha|^2$ or $1 - |\alpha|^2$. Notice now that the requirement for a quantum state to have length one is just a way to make sure that probabilities over all possible outcomes add to one.

More generally, if we write our N - dimensional quantum state as a vector

of N complex numbers,

$$\begin{bmatrix} v_0 \\ v_1 \\ \vdots \\ v_{N-1} \end{bmatrix}, \quad v_j \in \mathbb{C}, \forall j, \quad (13)$$

and measure this state in the natural standard set of vectors,

$$\begin{bmatrix} 1 \\ 0 \\ \vdots \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ \vdots \end{bmatrix}, \dots, \begin{bmatrix} 0 \\ \vdots \\ 1 \end{bmatrix}, \quad (14)$$

then the i -th outcome is given by the modulus of the i -th component of the state vector squared $|v_i|^2$. After this measurement, the new state is given by a wave function with a single nonzero component: only the i -th component is one $v_i = 1, v_{j \neq i} = 0 \forall j$.

2.5 Postulate 4: Compositions of quantum systems

Let's now build larger quantum systems from smaller ones. For at least two quantum systems, we can talk about uniquely quantum interactions, such as quantum entanglement.

Postulate 4. *When combining two quantum systems whose descriptions exist in spaces $\mathbb{C}^N$ and $\mathbb{C}^M$, the joint system is described by a tensor product of these two systems, $\mathbb{C}^N \otimes \mathbb{C}^M$.*

Definition 10. *A tensor (Kronecker) product, $\otimes$, of two arrays A (dimension $m \times n$) and B (dimension $p \times q$) will produce a larger block array C (with dimension $mp \times nq$). (Imagine C is made up of blocks where each element of A is multiplied by a copy of B).*

Consider the tensor product of two qubit wavefunctions,

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} \otimes \begin{bmatrix} \gamma \\ \delta \end{bmatrix} = \begin{bmatrix} \alpha\gamma \\ \alpha\delta \\ \beta\gamma \\ \beta\delta \end{bmatrix}. \quad (15)$$

In this two-qubit example, the composite system has four possible basis states,

$$|00\rangle := |0\rangle \otimes |0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}. \quad (16)$$

$$|01\rangle = |0\rangle \otimes |1\rangle = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad (17)$$

$$|10\rangle = |1\rangle \otimes |0\rangle = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad (18)$$

$$|11\rangle = |1\rangle \otimes |1\rangle = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}. \quad (19)$$

You can make the basis for any N number of qubits in this way, and this is known as the computational basis. What this means is that we can expand any wavefunction on these vectors, and for two qubits, we can write

$$|\Phi\rangle = \alpha_0|00\rangle + \alpha_1|01\rangle + \alpha_2|10\rangle + \alpha_3|11\rangle. \quad (20)$$

Exercise: In the example above, what is the probability of measuring ‘01’?

For the parameters $\alpha_0 = 1, \alpha_1 = i, \alpha_2 = -1, \alpha_3 = -i$, the two-qubit state can be expressed in a product state,

$$|\Phi\rangle = \frac{1}{2} \left(|0\rangle^{(1)} - |1\rangle^{(1)} \right) \left(|0\rangle^{(2)} + |1\rangle^{(2)} \right), \quad (21)$$

where superscripts index the two different subsystems.

Finally, we can now discuss ‘spooky action at a distance’ - namely that two quantum systems can have states which are correlated, and these correlations cannot be explained non-quantumly.

Definition 11 (Entanglement). *A joint state of two systems is called entangled if it cannot be written as a tensor product of quantum states on the individual systems.*

*Exercise:** Show that the output state $CX_{1,2}(H_1 \otimes I_2)|00\rangle$ is $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Can this be written as a tensor product of single qubit states?

Exercise: Consider the state $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Suppose I measure qubit 1 only. What are the probabilities of seeing 0 or 1 on qubit 1? After measuring qubit 1, can I deduce any information about the state of qubit 2 without measuring it?

Notice that the state $CX_{1,2}(H_1 \otimes I_2)|00\rangle$ uses a CX gate to prepare it. Often, a CX is called an entangling gate because $|0\rangle\langle 0| \otimes I_2 + |1\rangle\langle 1| \otimes X_2$ cannot be expressed as a tensor product of two single-qubit operators. In particular, the CX gate says apply X to qubit 2 if qubit 1 is in $|1\rangle$.

3 Extra for experts (not examinable)

We will now present the previous postulates in a more general manner, but this won't be covered in lectures as linear algebra is not a prerequisite for this course.

3.1 Postulates revisited

We can discuss bras and kets more formally with using concepts from linear algebra. If the quantum state vectors $|\phi_i\rangle$ form a basis for $\mathbb{C}^N$, then an arbitrary state in $\mathbb{C}^N$ can be written as a superposition of these basis states,

$$|\Phi\rangle = \sum_{i=1}^{2^N} \alpha_i |\phi_i\rangle, \quad \sum_{i=1}^{2^N} |\alpha_i|^2 = 1. \quad (22)$$

In the above, α_i are complex, scalar-valued coefficients and $*$ denotes complex conjugation. Corresponding to kets, one can define a (dual) elements called bra-vectors $\langle \cdot |$ as,

$$\langle \Phi | = \sum_{i=1}^n \alpha_i^* \langle \phi_i |, \quad (23)$$

where $*$ denotes a complex conjugation. Here the dual bra $\langle \phi_i |$ can be thought of as a 'row vector' with all its entries conjugated.

Mathematically, the bracket $\langle \Phi | \Psi \rangle$ is called an inner product.

Definition 12 (Inner product). *An inner product between two column vectors v, w is given by multiplying the transpose of v^* with w . In Dirac bra-ket notation, we just write $\langle v|w\rangle$. Here are some other properties of inner products:*

$$\langle \Phi|\Psi\rangle \in \mathbb{C}, \quad (24)$$

$$\langle \Phi|\Phi\rangle \geq 0, \langle \Phi|\Phi\rangle = 0 \iff \Phi \equiv 0, \quad (25)$$

$$\langle \Phi|\Psi\rangle = \langle \Phi|\Psi\rangle^* \in \mathbb{C}, \quad (26)$$

$$\langle \Phi|\alpha_1\Psi_1 + \alpha_2\Psi_2\rangle = \alpha_1\langle \Phi|\Psi_1\rangle + \alpha_2\langle \Phi|\Psi_2\rangle \quad (27)$$

The first property says that the inner product results in a complex number. The second property is used to define that all states must have a positive length i.e. the length of a quantum state is zero if and only if $|\Phi\rangle \equiv 0$. Generally $\langle \Phi|\Phi\rangle = 1$ if $|\Phi\rangle$ represents a normalized state, and $\langle \Phi|\Psi\rangle = 0$ if the two normalized states are orthogonal i.e. they have no overlap. The third and fourth properties say that the inner product is linear in the first argument, and anti-linear in the second argument.)

More generally, the physically measured properties of a system, for example, can include energy, momentum, position, spin. Since these properties can be discrete or continuous, we need a measurement postulate that places both discrete and continuous phenomena on an equal footing [7]. I'm going to restate the measurement postulate the hard way using some concepts of eigenvalues and eigenvectors from linear algebra, which are not examinable. Let us remind ourselves what an eigenvalue and eigenvectors are.

Definition 13. *Let $\hat{A}$ be a matrix and let $\{|\phi_n\rangle\}$ be vectors, and let a_n be scalars. If $\hat{A}|\phi_n\rangle = a_n|\phi_n\rangle$, then we say a_n is a eigenvalue of $\hat{A}$ with eigenvector $|\phi_n\rangle$ for each n . If $\hat{A}$ is Hermitian ($\hat{A}^\dagger = \hat{A}$) then by the spectral representation theorem, $\hat{A}$ has (i) real eigenvalues, (ii) eigenvectors corresponding to distinct eigenvalues are orthogonal, and (iii) there exists an orthogonal basis of the whole space, consisting of eigenvectors of $\hat{A}$.*

Postulate 5 (Physical observables). *(Hard way) Each physically observable quantity of a system is associated with a linear Hermitian operator $\hat{A}$, and the observed values of this quantity are the eigenvalues $\{a_n\}$ of $\hat{A}$. Let a quantum state $|\Psi\rangle$ be expanded in the eigenstates of $\{|\phi_n\rangle\}$ of $\hat{A}$ with coefficients $\{\alpha_n\}$*

$$|\Psi\rangle = \sum_n \alpha_n |\phi_n\rangle. \quad (28)$$

Then Born's rule gives the probabilities of measuring an outcome a_n in that basis,

$$\text{Prob}(a_n) := |\langle a_n | \Psi \rangle|^2 = |\alpha_n|^2. \quad (29)$$

After measurement of the system in $|\Psi\rangle$, the quantum state collapses to $|a_n\rangle$. Over an ensemble of many measurements of $|\Psi\rangle$ in the eigenbasis of $\hat{A}$, the average value of $\hat{A}$ is

$$\langle \hat{A} \rangle := \langle \Psi | \hat{A} | \Psi \rangle. \quad (30)$$

Throughout this lecture notes, we have always measured qubits in the ‘computational basis’ i.e. eigenstates of the Z operator. Note that all Pauli matrices I, X, Y, Z are both unitary (valid quantum gates) but also Hermitian (valid physical observables). Since Z is Hermitian, it has real eigenvalues, and the eigenvectors of Z form an orthogonal basis. You can check that the eigenstates of Z are $|0\rangle, |1\rangle$ with eigenvalues $1, -1$. They are orthogonal since $\langle 0|0\rangle = \langle 1|1\rangle = 1$ but $\langle 0|1\rangle = \langle 1|0\rangle = 0$.

What we’re actually doing the ‘easy’ way is that we are measuring the qubit $|\Psi\rangle$ in a basis defined by the eigenstates of Z . By convention, we could call the 1 eigenvalue of Z the ground state, and the -1 eigenvalue of Z the excited state (or the other way around). Since Z only has 2 eigenvalues, it doesn’t matter what convention is used - the observed quantity is still a binary outcome.

The previous exercises show that any Hermitian operator can be written as the sum over the outer product of its eigenstates $\hat{A} := \sum_n a_n |a_n\rangle \langle a_n|$.

Exercise: Check that $|+\rangle := H|0\rangle$ and $|-\rangle := H|1\rangle$ are eigenstates of X , with eigenvalues of 1 and -1 .

Exercise: How would you measure a general single qubit quantum state Ψ in the eigenbasis of X ? What are the probabilities of seeing $|\pm\rangle$ states in terms of α, β ?

A consequence of using Hermitian matrices to represent physically measurable quantities is that we cannot know everything about a quantum system simultaneously. In fact, we can only simultaneously know about physical properties of a quantum system only if the matrices (operators) associated with them commute.

Definition 14. The commutator of two square matrices A and B is

$$[A, B] = AB - BA. \quad (31)$$

The anti-commutator of two square matrices A and B is

$$\{A, B\} = AB + BA. \quad (32)$$

Using the notation above, two quantum operators are said to commute if $[\hat{A}, \hat{B}] = 0$, in which case $\hat{A}, \hat{B}$ have a complete set of vectors that are eigenvectors of both $\hat{A}$ and $\hat{B}$. A complete, commuting set of matrices means that for each eigenvalue, there is no more than one eigenvector. In this case, a complete, commuting set of matrices is equivalent to a single operator with a non-degenerate spectrum [7]. Namely, if the matrix representation of two quantum operators commute, you can measure in some basis where the system will ‘collapse’ into a valid eigenstate and you can obtain an eigenvalue / measurement for both physical properties. If they do not commute, then no such measurement basis exists - learning about one physical property means giving up all information about the other physical property.

Exercise: Show X and Z operators for a single qubit do not commute.

3.2 Noisy quantum states

Finally, we haven’t talked about quantum systems that are not well-isolated, and actually interact with the rest of the universe. The interaction of a quantum state with noise i.e. uncontrolled loss of information to the rest of the universe, can lead to *classical* statistical mixtures of quantum states.

The outer product $|0\rangle\langle 0|$ example shows that any quantum state $|\Psi\rangle$ can be associated with a matrix $|\Psi\rangle\langle\Psi|$. We call these matrices a *density operator* and these are typically used in the study of quantum systems subject to noise. In particular, these matrices generalize a quantum state to a *classical*, probabilistic mixture of *many* quantum states.

Definition 15 (Density operators). A density operator ρ represents a convex linear combination of a set of quantum states $\{|\Psi_i\rangle\}$,

$$\rho := \sum_i c_i |\Psi_i\rangle\langle\Psi_i|, \quad \sum_i c_i = 1, c_i \in [0, 1], \quad (33)$$

where c_i represents a classical probability for the quantum state $|\Psi_i\rangle$ in the mixture. Valid classical statistical mixtures of pure quantum states mean that all ρ satisfy $\rho > 0, \rho^\dagger = \rho$ and the sum of diagonal entries of ρ are unity, i.e. $\text{Tr}[\rho] = 1$. If there is only one quantum state ($i = 1$), then $\rho = |\Psi\rangle\langle\Psi|$. A quantum state is pure if and only $\text{Tr}[\rho^2] = 1$, and is called mixed otherwise.

In the noisy setting, the expectation value of any observable $\hat{A}$ is given by $\text{Tr}[\rho\hat{A}]$. The (Born) probability of a measurement outcome a_n given by $\text{Tr}[\rho M]$, which collapses to the state $\frac{M\rho M^\dagger}{\text{Tr}[\rho M]}$, for a projector $M = |a_n\rangle\langle a_n|$, where a_n is the eigenvalue of eigenstate $|a_n\rangle$ of $\hat{A}$.

In particular, we can explain the Bloch sphere representation of the qubit. For any ρ for n -qubits, we can expand this density operator in a basis formed by Pauli matrices $\{I, X, Y, Z\}^n$,

$$\rho := \frac{1}{2^n} \sum_{i=0}^{2^n-1} c_i P_i, \quad P_i \in \{I, X, Y, Z\}^n. \quad (34)$$

For a single qubit, this gives

$$\rho := \frac{1}{2} (c_0 I + c_1 X + c_2 Y + c_3 Z), \quad (35)$$

but we can reduce one unknown parameter c_0 by using the properties of ρ . Since by definition, $\text{Tr}(\rho) = 1$, we get $\frac{1}{2}\text{Tr}(c_0 I + c_1 X + c_2 Y + c_3 Z) = \frac{c_0}{2}\text{Tr}(I) = c_0 = 1$. Then, the remaining vector $\vec{c} = [c_1, c_2, c_3]$ is called a Bloch vector, and represents values along a sphere where the coordinate system represents directions along X , Y and Z Pauli operators.

A visual Bloch sphere is shown in Fig. 2. Notice that pure states $\text{Tr}(\rho^2) = 1$ implies that

$$\frac{1}{4}\text{Tr}[(I + c_1 X + c_2 Y + c_3 Z)(I + c_1 X + c_2 Y + c_3 Z)] = \frac{1}{2}(1 + c_1^2 + c_2^2 + c_3^2) = 1, \quad (36)$$

which shows $c_1^2 + c_2^2 + c_3^2 = 1$. Namely, pure states are unit vectors on the Bloch sphere and lie on the surface of Fig. 2. Meanwhile, mixed states $\text{Tr}(\rho^2) < 1$ lie in the bulk of the sphere.

3.3 No cloning theorem

This subsection is not examinable but talks about a fairly important result in quantum information that is also easy to prove based on the tools we've just learned about. This landmark result [14] showed that the linearity of quantum mechanics prevents perfect copying of an *unknown* quantum state. The result is called the quantum no-cloning theorem.

To clone a single qubit, we are asking if there is a unitary U that takes $|0\rangle \otimes |\Psi\rangle \rightarrow |\Psi\rangle \otimes |\Psi\rangle$. Since the input looks like,

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} \otimes \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} \alpha \\ 0 \\ \beta \\ 0 \end{bmatrix} \quad (37)$$

and the output looks like,

$$\begin{bmatrix} \alpha \\ \beta \end{bmatrix} \otimes \begin{bmatrix} \alpha \\ \beta \end{bmatrix} = \begin{bmatrix} \alpha\alpha \\ \alpha\beta \\ \alpha\beta \\ \beta\beta \end{bmatrix}, \quad (38)$$

any transformation between them cannot be linear.

To clarify, you can prepare a known quantum state $|\Psi\rangle$ repeatedly. However if someone was to give you $|\Psi\rangle$ but you don't know what it is, then you cannot make a copy of it.

We now provide a standard proof of no-cloning for all quantum states.

Theorem 1 (No cloning). *Let $|\Psi\rangle_A$ and $|\Psi\rangle_B$ be identical states in two systems $A \otimes B$. Let $|b\rangle_B$ be some normalised state in B . There is no unitary operator U acting on $A \otimes B$ such that*

$$U(|\Psi\rangle_A \otimes |b\rangle_B) = e^{-if(\theta,b)} |\Psi\rangle_A \otimes |\Psi\rangle_B, \quad (39)$$

for any choice of a real number $f(\theta, b)$ potentially depending on b or θ .

Proof. We use a proof by contradiction: we assume U exists and then find a logical contradiction. The contradiction tells us that our original assumption (U exists) cannot be true. To do this, we will pick any two states in first system, say $|v\rangle_A$ and $|w\rangle_A$ and suppose they overlap by a complex number $c := \langle w|v\rangle$. To each of these states, we append a known (normalised) state $|b\rangle_B$ in B (you can think of this as $|0\rangle_B$ for $b = 0$). My initial states are,

$$|V_0\rangle = |v\rangle_A \otimes |b\rangle_B \quad (40)$$

$$|W_0\rangle = |w\rangle_A \otimes |b\rangle_B, \quad (41)$$

and the initial overlap is

$$\langle W_0|V_0\rangle = (\langle w|_A \otimes \langle b|_B) (|v\rangle_A \otimes |b\rangle_B) \quad (42)$$

$$= \langle w|v\rangle_A \otimes \langle b|b\rangle_B \quad (43)$$

$$= \langle w|v\rangle \langle b|b\rangle \quad (44)$$

$$= c, \quad (45)$$

since $\langle b|b\rangle = 1$. Now, we evolve both states by U ,

$$U|V_0\rangle = e^{-if(\theta_1,b)}|v\rangle_A \otimes |v\rangle_B \quad (46)$$

$$U|W_0\rangle = e^{-if(\theta_2,b)}|w\rangle_A \otimes |w\rangle_B, \quad (47)$$

which clones both states. However, if we substitute $U^\dagger U = I$, the overlap becomes,

$$\langle W_0|V_0\rangle = \langle W_0|I|V_0\rangle \quad (48)$$

$$= \langle W_0|U^\dagger U|V_0\rangle \quad (49)$$

$$= \left(\langle W_0|U^\dagger\right) U|V_0\rangle \quad (50)$$

$$= \left(\langle e^{+if(\theta_2,b)}\langle w|_A \otimes \langle w|_B\right) U|V_0\rangle \quad (51)$$

$$= \left(\langle e^{+if(\theta_2,b)}\langle w|_A \otimes \langle w|_B\right) \left(e^{-if(\theta_1,b)}|v\rangle_A \otimes |v\rangle_B\right) \quad (52)$$

$$= e^{+if(\theta_2,b)-if(\theta_1,b)} (\langle w|v\rangle_A \otimes \langle w|v\rangle_B) \quad (53)$$

$$= e^{+if(\theta_2,b)-if(\theta_1,b)} c^2. \quad (54)$$

In the first three lines, we are using $I = U^\dagger U$ and rearranging terms. In the fourth and fifth step, we substitute the definition U to clone $|w\rangle$ and $|v\rangle$. In the sixth step, we multiply over the tensor product and use the definition of c . Notice we can always pick θ_1, θ_2 such that the $f(\theta_2, b) - f(\theta_1, b) \equiv 0$. So now we have a contradiction since,

$$\langle W_0|V_0\rangle = c = c^2, \quad (55)$$

which can only hold if $c = 0, 1$ (cloned states $|v\rangle, |w\rangle$ are orthogonal) but does not hold for all values of c . This contradiction means U does not exist generally, i.e. that you cannot clone arbitrary unknown quantum states.

As Ref.[2] discusses, the classical logical operations of AND, OR, XOR, NAND or FANOUT cannot be used directly in quantum computing. The AND, OR, XOR and NAND gates are not reversible i.e. they are not unitary gates. The FANOUT gate would not be allowed in quantum computing since it involves the duplication, or cloning, of a state; this would violate the no-cloning theorem which is presented in Section 3.3. Of course, all classical logical operations can be implemented on a quantum computer using reversible operations without cloning. I haven't introduced these gates since we can achieve universality without them.

x	f_1	f_2	f_3	f_4
0	0	1	0	1
1	0	1	1	0

Table 2: Output for the functions f_1, f_2, f_3, f_4 , which are constant for $A = \{f_1, f_2\}$ and balanced for $B = \{f_3, f_4\}$.

4 Quantum algorithms

Here is the computational task that we will solve, first classically, then quantumly.

4.1 Constant vs. balanced oracle problem

Definition 16 (Constant vs. balanced boolean functions). *A boolean function $f(x)$ takes as input a binary string of n bits, $x \in \{0, 1\}^n$. It is called ‘constant’ if it returns the same value for all choices of x . It is called ‘balanced’ if it returns 0 for half the possible values of x , and returns 1 for the remaining half.*

Notice that for a single $n = 1$ bit boolean function, the possibilities of all possible functions illustrated in Table 2. In this case, these functions divide into 2 groups, $A = \{f_1, f_2\}$ and $B = \{f_3, f_4\}$. For $n = 2$ bit boolean functions, as you will see in the lab, there are 6 balanced functions and 2 constant functions.

Based on these definitions, we consider the following task.

Definition 17 (Constant vs. balanced oracle problem ($n = 1$ bit)). *Let f be an input function: $\{0, 1\} \rightarrow \{0, 1\}$. Output if f is constant or balanced.*

In other words, given two groups $A = \{f_1, f_2\}$ and $B = \{f_3, f_4\}$ and an oracle that implements one of these functions, how many queries are required to decide if the oracle is implementing a function from group A or B ? (Note that functions in group A are *constant*. Functions of group B are *balanced*).

Classically, if inputs are one bit, then one must query f_i twice to distinguish the constant function from the balanced function. For an n -bit input, one must query $2^{n-1} + 1$ times.

We are almost ready to study a quantum algorithm to solve this task. The trick for solving this task using quantum computation relies on the phase kick-back effect. Next, we discuss this phase kickback effect, and then we will see how the phase kickback effect is used in Deutsch's algorithm.

4.2 Phase kickback

Let's do an example calculation based on the tools we have. Suppose I have 2 qubits. If I prepare both qubits in $|00\rangle, |01\rangle, |10\rangle$ or $|11\rangle$, I basically have two classical bits of information. Suppose I want to have all the qubits to be in a superposition of $|0\rangle$ and $|1\rangle$ states. How do I do this? For two qubits, I could apply a Hadamard to both qubits,

$$|++\rangle = (H_1 \otimes H_2) (|0\rangle \otimes |1\rangle) = |+\rangle|+\rangle, \text{ or,} \quad (56)$$

$$|+-\rangle = (H_1 \otimes H_2) (|0\rangle \otimes |1\rangle) = |+\rangle|-\rangle. \quad (57)$$

Recalling that,

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad (58)$$

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}, \quad (59)$$

we can show, similarly, use the definitions of $|\pm\rangle$ to write out $|+-\rangle$ in zeroes and ones,

$$|++\rangle = |+\rangle \otimes |+\rangle = \frac{1}{2} (|00\rangle + |01\rangle + |10\rangle + |11\rangle) \quad (60)$$

$$|+-\rangle = |+\rangle \otimes |-\rangle = \frac{1}{2} (|00\rangle - |01\rangle + |10\rangle - |11\rangle). \quad (61)$$

We can read-off the Born probabilities from looking at $|++\rangle$ and $|+-\rangle$ i.e. we will measure outcomes 00, 01, 10, 11 with 25% probability each. Measuring in this computational basis doesn't help me distinguish between $|++\rangle$ and $|+-\rangle$.

We will now see how this relative phase difference in $|+-\rangle$ helps the quantum states of qubit 1 and qubit 2 to *interfere* with each other. Let's apply a CX gate to these superpositions. What we will find is that,

$$CX|++\rangle = |++\rangle \quad (62)$$

$$CX|+-\rangle = |--\rangle, \quad (63)$$

where the state $|++\rangle$ is unchanged due to the action of the CX gate, but the state $|+-\rangle$ changes to $|- -\rangle$. The latter case shows one example of a phase-kickback. The CX gate, by definition, flips qubit 2 if qubit 1 is $|1\rangle$. Remarkably, qubit 2 in $CX|+-\rangle$ is unchanged and always remains in $|-\rangle$. Meanwhile, the relative phase of the superposition states in qubit 1 changes from $|+\rangle$ to $|-\rangle$.

For the rest of this subsection, we will focus on just deriving $CX|+-\rangle = |- -\rangle$ using the tools we have learned. A neat trick is to notice that,

$$CX := |0\rangle\langle 0| \otimes I_2 + |1\rangle\langle 1| \otimes X_2, \quad (64)$$

Applying the CX gate to this superposition gives,

$$CX|+-\rangle = \frac{CX}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle) \quad (65)$$

$$= \frac{1}{2}(CX|00\rangle - CX|01\rangle + CX|10\rangle - CX|11\rangle) \quad (66)$$

$$= \frac{1}{2}(|00\rangle - |01\rangle + |11\rangle - |10\rangle) \quad (67)$$

$$= \frac{1}{2}(|00\rangle - |01\rangle - |10\rangle + |11\rangle) \quad (68)$$

$$= \frac{1}{2}(|0\rangle \otimes (|0\rangle - |1\rangle) - |1\rangle \otimes (|0\rangle - |1\rangle)) \quad (69)$$

$$= \frac{1}{2}(|0\rangle|-\rangle - |1\rangle|-\rangle) \quad (70)$$

In the first line, I used definitions of CX and $|+-\rangle$. In the second line, I applied the CX to each two-qubit state in the superposition by linearity. In the third line, I flipped the second qubit if the first qubit was one. The fourth line is a rearrangement of terms. The fifth line is grouping terms by the states of qubit 1. The last line is substituting the definition of $|-\rangle$ to make things easier to see.

Notice that we started out with a $|-\rangle$ on the second qubit, and the CX also physically only acts on the second qubit. However, I've somehow "kicked back" the phase (-1) which accrued on the *second* qubit to the superposition on the *first* qubit. Indeed,

$$CX|+-\rangle = \frac{1}{2}(|0\rangle|-\rangle - |1\rangle|-\rangle) = |- -\rangle, \quad (71)$$

shows that the relative phase on the second qubit hasn't changed but the relative phase on the first qubit has.

Exercise: Show that $CX|+-\rangle = |- -\rangle$ induces a phase on qubit 1 by acting on qubit 2.

Compare this to the case where we don't entangle the qubits,

$$(I_1 \otimes X_2)|+-\rangle = \frac{(I_1 \otimes X_2)}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle), \quad (72)$$

$$= \frac{1}{2}(I|0\rangle \otimes X|0\rangle - I|0\rangle \otimes X|1\rangle + I|1\rangle \otimes X|0\rangle - I|1\rangle \otimes X|1\rangle), \quad (73)$$

$$= \frac{1}{2}(|01\rangle - |00\rangle + |11\rangle - |10\rangle), \quad (74)$$

$$= -|+-\rangle, \quad (75)$$

where the relative phases of each qubit is unchanged, but the state has inherited a *global* phase of -1 .

4.3 Deutsch's algorithm

Let's now solve this task with a quantum algorithm.

Let f be an input function: $\{0, 1\} \rightarrow \{0, 1\}$. Output if f is constant or balanced.

Claim: For an quantum oracle that implements f as the unitary U_f ,

$$U_f(|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \oplus f(x)\rangle, \quad x, y \in \{0, 1\}, \quad (76)$$

this task is solved by a single query by measuring,

$$|D\rangle := (H_1 \otimes H_2) U_f (H_1 \otimes H_2) |01\rangle. \quad (77)$$

If f is constant, we measure 0 on the first qubit with probability 100%. Else, if f is balanced, we measure 1 on the first qubit.

First, let's construct the oracle to query. The construction of the oracle, or the query gates, is part of the preparation of the input, not part of finding a solution. In particular, the functions f_1, f_2, f_3, f_4 represent non unitary transformations on a single bit, but they can be represented as a unitary U_{f_i} on two bits i.e. by adding an 'ancillary' bit that we will eventually ignore. Since we can only represent unitary transformations as gates on a quantum computer, it's easier to work in this 2-bit setting.

Consider the following 2-bit boolean functions that have the same truth table as Table 2. These four functions take as input two classical bits, $x, y \in [0, 1]$, and output two classical bits,

$$f_1(x, y) = (x, y), \quad (78)$$

$$f_2(x, y) = (x, \neg y), \quad (79)$$

$$f_3(x, y) = (x, x \oplus y), \quad (80)$$

$$f_4(x, y) = (x, x \oplus \neg y). \quad (81)$$

Here, we negate a bit by flipping it $\neg 1 = 0, \neg 0 = 1$. We also add two bits, $\oplus$, where addition happens ‘on a circle’ with 2 points, such that we get regular addition for the cases $0 \oplus 0 = 0$ and $0 \oplus 1 = 1 \oplus 0 = 1$ but then we set $1 \oplus 1 = 0$. In this setting, notice you can ignore the first output bit, and set $y = 0$ to yield the same truth table Table 2. Setting $y = 1$ doesn’t change whether a function is constant or balanced, but just globally negates Table 2. You can check that the following quantum operations,

$$U_{f_1} := I_1 \otimes I_2, \quad (82)$$

$$U_{f_2} := I_1 \otimes X_2, \quad (83)$$

$$U_{f_3} := CX = \begin{bmatrix} I & 0 \\ 0 & X \end{bmatrix}, \quad (84)$$

$$U_{f_4} := \begin{bmatrix} X & 0 \\ 0 & I \end{bmatrix}, \quad (85)$$

are implement the boolean functions f_1, f_2, f_3, f_4 on two qubits $|0\rangle|1\rangle$. Here U_{f_4} is a gate that applies X to qubit 2 if qubit 1 is in $|0\rangle$. You can also see that each of these unitaries satisfy $U_{f_i}(|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \oplus f_i(x)\rangle$, $x, y \in \{0, 1\}$ for all $i = 1, 2, 3, 4$.

Having constructed the oracle, we now see what happens when these unitaries are applied to a specific quantum state, $|+-\rangle$. In particular, if U_{f_1} or U_{f_2} for constant functions acts on the superposition $|+-\rangle$, then there is no entanglement between the qubits and the phase kickback effect does not occur. We get,

$$U_{f_1}|+-\rangle = |+-\rangle \quad (86)$$

$$U_{f_2}|+-\rangle = -|+-\rangle. \quad (87)$$

In contrast, if U_{f_3} or U_{f_4} for balanced functions act on $|+-\rangle$, then we use a CX between the two qubits. The phase kickback occurs, and we get,

$$U_{f_3}|+-\rangle = |--\rangle \quad (88)$$

$$U_{f_4}|+-\rangle = -|--\rangle. \quad (89)$$

Remember that the global phase of -1 between the two states $U_{f_1}|+-\rangle$ and $U_{f_2}|+-\rangle$ won't manifest in measurement outcomes because measurement probabilities do not change. To see this, note that once we apply the final set of Hadamard gates, we get

$$|D_1\rangle = (H_1 \otimes H_2)U_{f_1}|+-\rangle = (H_1 \otimes H_2)|+-\rangle = |01\rangle \quad (90)$$

$$|D_2\rangle = (H_1 \otimes H_2)U_{f_2}|+-\rangle = -(H_1 \otimes H_2)|+-\rangle = -|01\rangle, \quad (91)$$

so measuring either of these states $|D_1\rangle, |D_2\rangle$.

Similarly that the global phase of -1 between the two states $U_{f_3}|+-\rangle$ and $U_{f_4}|+-\rangle$ won't make a difference to measurement outcomes,

$$|D_3\rangle = (H_1 \otimes H_2)U_{f_3}|+-\rangle = (H_1 \otimes H_2)|--\rangle = |11\rangle \quad (92)$$

$$|D_4\rangle = (H_1 \otimes H_2)U_{f_4}|+-\rangle = -(H_1 \otimes H_2)|--\rangle = -|11\rangle, \quad (93)$$

so measuring either of these states $|D_3\rangle, |D_4\rangle$.

*Exercise**: Verify for yourself the results above i.e. compute $|D_i\rangle$ for each $i = 1, 2, 3, 4$, and state the measurement outcomes and their probabilities. What are the measurement outcomes and measurement probabilities if f is constant? What are the measurement outcomes and measurement probabilities if f is balanced?

*Exercise**: How many quantum queries do I need to solve my task? How many classical queries do I need to solve this task?

The above exercises should reveal that if f is constant, one obtains the outcome 01 with 100% probability. Instead if f is balanced, one obtains the outcome 11 with 100% probability. We need a single measurement to distinguish between constant and balanced functions, so the number of quantum queries is one.

4.4 Extending to Deutsch-Jozsa algorithm

We will now extend this problem to n -bit strings for $n > 1$. In the assignment, you will write a program to confirm that you only need one quantum query to determine constant vs balanced function for any n . You can find a great online tutorial and code in Ref. [15] and we will largely follow this approach.

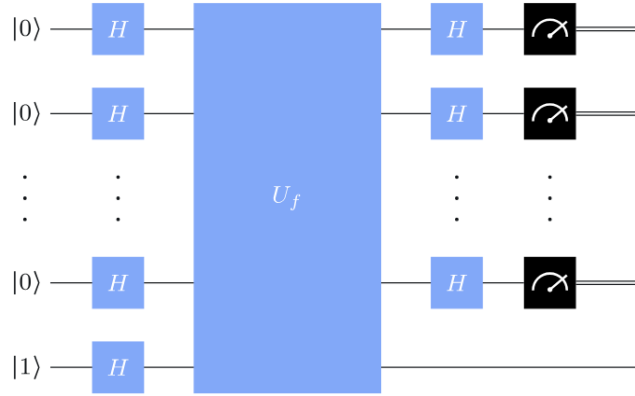


Figure 4: Depiction of the Deutsch-Jozsa algorithm for n -bit input as a quantum circuit containing $n + 1$ qubits. A single qubit is depicted as a solid horizontal wire, with its initial state shown as a ket on left, and evolving in time from left to right. Boxes represent single or multi-qubit gates. Measurement of a qubit is given as a black dial, with probability of ‘0’ and ‘1’ outcomes given by the Born rule. Image credit: [15]

Deutsch Jozsa Algorithm for Constant vs. Balanced f : Let f be an input function: $\{0,1\}^n \rightarrow \{0,1\}$. Output if f is constant, or balanced. It is promised that f will be either constant or balanced.

Claim: For an quantum oracle that implements f as the unitary U_f ,

$$U_f (|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \oplus f(x)\rangle, \quad x \in \{0,1\}^n, y \in \{0,1\}, \quad (94)$$

this task is solved by a single query using the quantum algorithm depicted in Fig. 4. If f is constant, we measure 0 on the first n qubits, else f is balanced.

Please use the following programming notebook for this laboratory: [link to workbook](#). The majority of the work will be around figuring out what U_f looks like for $n > 1$.

The proof for why Deutsch-Jozsa algorithm works for $n > 1$ has the same intuition as a $n = 1$ case we derived in the previous section, but a little more math. The proof for $n > 1$ is not examinable in this short lecture series but you can play around with the lab book to verify that this is indeed the case.

For completeness, I will sketch the main mathematical proof and the tricks we use here. The first trick is that we write tensor products of many qubits in a short hand using a binary string $x = x_1x_2\dots x_n$,

$$|x\rangle := |x_1x_2\dots x_n\rangle = |x_1\rangle|x_2\rangle\dots|x_n\rangle. \quad (95)$$

The second trick is to write a tensor product of Hadamards on n qubits as,

$$H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle, \quad (96)$$

where $x \cdot y$ is a dot product between n -bit binary strings x and y with addition mod 2, $x_1y_1 \oplus \dots \oplus x_ny_n$. This dot product is 0 if normal addition $x_1y_1 + \dots + x_ny_n$ is even, or 1 if the normal addition is odd.

The third trick is to re-write the oracle U_f in a more convenient form, at least for analysis, if not for programming a quantum circuit. It is true that for any $a, b \in \{0, 1\}$, the quantum state $|a \oplus b\rangle = X^b|a\rangle$. Substituting this into the oracle gives,

$$U_f(|x\rangle|y\rangle) = |x\rangle|y \oplus f(x)\rangle = |x\rangle \left(X^{f(x)}|y\rangle \right). \quad (97)$$

Since this is true for $|y\rangle = |0\rangle$ and $|y\rangle = |1\rangle$, it must also be true for any superpositions of these states including the case $|y\rangle = |-\rangle$. Indeed when we use this oracle on the state $|x\rangle|-\rangle$, we can see the phase kickback phenomenon more clearly,

$$U_f(|x\rangle|-\rangle) = |x\rangle \left(X^{f(x)}|-\rangle \right), \quad (98)$$

$$= |x\rangle \left((-1)^{f(x)}|-\rangle \right), \quad (99)$$

$$= (-1)^{f(x)}|x\rangle|-\rangle, \quad (100)$$

where $X^{f(x)}|-\rangle = (-1)^{f(x)}|-\rangle$ since $X|-\rangle = -|-\rangle$, and the final step shows the ‘kickback’ of the phase $(-1)^{f(x)}$.

Let’s now follow Fig. 4 and put it all together. We write the first layer of Hadamards acting on all $|0\rangle$ qubits in Fig. 4 to give the superposition of all possible bitstrings,

$$(H^{\otimes n}|0\rangle^{\otimes n}) \otimes H|1\rangle = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} |x\rangle|-\rangle. \quad (101)$$

Next, we act on this state with U_f ,

$$U_f (H^{\otimes n+1} |0\rangle^{\otimes n} |1\rangle) = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} U_f(|x\rangle |-\rangle) \quad (102)$$

$$= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} |x\rangle |-\rangle, \quad (103)$$

which gives the phase kickback. Lastly, we apply the Hadamards on the first n qubits,

$$(H^{\otimes n} \otimes I) U_f (H^{\otimes n+1} |0\rangle^{\otimes n} |1\rangle) = \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} (H^{\otimes n} |x\rangle) |-\rangle, \quad (104)$$

$$= \frac{1}{\sqrt{2^n}} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \left(\frac{1}{\sqrt{2^n}} \sum_{y \in \{0,1\}^n} (-1)^{x \cdot y} |y\rangle \right) |-\rangle, \quad (105)$$

$$= \frac{1}{2^n} \sum_{x \in \{0,1\}^n} \sum_{y \in \{0,1\}^n} (-1)^{f(x) + x \cdot y} |y\rangle |-\rangle, \quad (106)$$

where substituted the definition of $H^{\otimes n} |x\rangle$ in second line, and rearranged terms in the final line.

Suppose we measure the all-zero case i.e. $|y\rangle \equiv |0\rangle^{\otimes n}$. In this case, $x \cdot y = 0$, and the Born probability of measuring this outcome is the single amplitude,

$$\left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \right|^2. \quad (107)$$

If f is constant, then either $f(x) = 0$ or $f(x) = 1$ for all bitstrings x . Therefore, the term $\frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} = \frac{\pm 2^n}{2^n} = \pm 1$ for constant f i.e. the square of this value is one. Physically this means that if f is constant, we will measure the all zero outcome with 100% probability. Similarly you can check that if f is balanced, then $f(x) = 0$ on half of all possible configurations x and $f(x) = 1$ on other half of the input space. These values of $f(x)$ on the full input space cancel, giving $\sum_{x \in \{0,1\}^n} (-1)^{f(x)} = 0$. Physically, this means that if f is balanced, there is 0% chance of measuring the all zero outcome on the first n qubits.

4.5 Random classical algorithm

Notice that the classical query cost is expensive only if we require absolutely perfect performance. If a classical procedure queries f in randomly then, in the case of a balanced function, it would have to be very unlucky to always draw the same bit value.

Suppose a *random* classical algorithm makes k queries randomly (independently) and then outputs “constant” if all queries are the same, or “balanced” otherwise. If f is constant, then we will be correct with 100% probability. If f is balanced, then we will be wrong with some probability ϵ .

Suppose I make k random queries. Let the first query be my reference. Then I would have to get the same output on $k - 1$ queries for me to incorrectly identify a true balanced function as being constant.

What is the probability that the classical algorithm fails? Well it’s a probability of seeing the same output $k - 1$ times, and seeing a different outcome on the k -query. The probability of seeing the same outcome $k - 1$ times is multiplicative, $\frac{1}{2} \times \frac{1}{2} \times \dots \times \frac{1}{2}$, which gives me a total probability of error of $\epsilon = (\frac{1}{2})^{k-1}$. The error probability for a random classical algorithm dies off pretty quickly as you increase the number of random classical queries k .

Exercise: Suppose one solves the constant vs. balance problem using a classical random algorithm and we are willing to tolerate ϵ error. Show that the number of classical queries required is now $O(\log_2(\frac{1}{\epsilon}))$. What can we conclude about quantum advantage for the constant vs. balanced oracle problem?

4.6 Bernstein-Varizani problem (not examinable)

It’s also notable that the same Deutsch Jozsa algorithm can solve another type of problem.

Deutsch Jozsa Algorithm for Bernstein-Varizani problem:

Let f be an input function: $\{0, 1\}^n \rightarrow \{0, 1\}$. It is promised that there is a binary string s which $f(x) = x \cdot s = x_1s_1 + x_2s_2 + \dots + x_ns_n \pmod{2}$. How many queries does it take to output s ?

Claim: For an quantum oracle that implements f , this task is solved by a single query using the quantum algorithm depicted in Fig. 4.

Again, the proof that Deutsch Jozsa Algorithm solves the Bernstein-Varizani problem is not examinable but you can play around in the lab work book to verify that this is indeed the case. The proof largely follows the intuition we have already discussed [15].

Congratulations, you've survived your first crash course on quantum computing!

References

- [1] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [2] J. D. Hidary, *Quantum Computing: An Applied Approach*. Cham, Switzerland: Springer International Publishing. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-030-83274-2>
- [3] “Supporting Code for Quantum Computing: An Applied Approach,” Mar. 2026, [Online; accessed 13. Mar. 2026]. [Online]. Available: <https://github.com/JackHidary/quantumcomputingbook>
- [4] “The Theory of Quantum Information,” Oct. 2020, [Online; accessed 12. Mar. 2026]. [Online]. Available: <https://cs.uwaterloo.ca/~watrous/TQI>
- [5] Qiskit, “Understanding Quantum Information & Computation With John Watrous,” Mar. 2026, [Online; accessed 12. Mar. 2026]. [Online]. Available: <https://www.youtube.com/playlist?list=PLOFEBzvs-VvqKKMXX4vbi4EB1uaErFMSO>
- [6] G. Bowman, *Essential Quantum Mechanics*. OUP Oxford, 2007.
- [7] L. E. Ballentine, *Quantum Mechanics: A Modern Development*. World Scientific Publishing Company, 1998.
- [8] P. Benioff, “The computer as a physical system: A microscopic quantum mechanical Hamiltonian model of computers as represented by Turing machines,” *J. Stat. Phys.*, vol. 22, no. 5, pp. 563–591, May 1980.
- [9] D. Deutsch, “Quantum theory, the church-turing principle and the universal quantum computer,” *Proceedings of the Royal Society of London. A. Mathematical and Physical Sciences*, vol. 400, no. 1818, pp. 97–117, 1985. [Online]. Available: <https://doi.org>
- [10] E. Bernstein and U. Vazirani, “Quantum Complexity Theory,” *SIAM J. Comput.*, vol. 26, no. 5, pp. 1411–1473, Oct. 1997.
- [11] A. Childs, “Lecture Notes on Quantum Algorithms,” Apr. 2025, [Online; accessed 12. Mar. 2026]. [Online]. Available: <https://www.cs.umd.edu/~amchilds/qa>

- [12] S. Jordan, “Quantum Algorithm Zoo,” Mar. 2026, [Online; accessed 12. Mar. 2026]. [Online]. Available: <https://quantumalgorithmzoo.org>
- [13] J. Watrous, “Quantum Computational Complexity,” *arXiv*, Apr. 2008.
- [14] W. K. Wootters and W. H. Zurek, “A single quantum cannot be cloned,” *Nature*, vol. 299, pp. 802–803, Oct. 1982.
- [15] “The Deutsch-Jozsa algorithm | IBM Quantum Learning,” Apr. 2026, [Online; accessed 15. Apr. 2026]. [Online]. Available: <https://quantum.cloud.ibm.com/learning/en/courses/fundamentals-of-quantum-algorithms/quantum-query-algorithms/deutsch-jozsa-algorithm>